



**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**
Membre de
HONORIS UNITED UNIVERSITIES

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI Casablanca

Filière : Ingénierie Informatique et Réseaux – 3IIR

Année universitaire : 2025/2026

RAPPORT DE PROJET DE FIN D'ANNÉE

FakeFinder

Système Intelligent de Détection du Hameçonnage par E-mail

Réalisé par :

Yahya KHALIFI

Elias CHAKROUN

Tuteur académique :

Mme Soukayna MAYARA

Date de soutenance :

18 Mai 2026

Remerciements

La réalisation de ce projet de fin d'année a été une expérience profondément enrichissante, tant sur le plan technique que professionnel. Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail de recherche et de développement.

Nos remerciements s'adressent en tout premier lieu à notre encadrante pédagogique, **Madame Soukayna MAYARA**. Son expertise technique, sa rigueur scientifique, ses conseils avisés et sa grande disponibilité ont été des atouts majeurs tout au long de ce projet. Ses directives nous ont permis de structurer notre travail de manière méthodique, de surmonter les obstacles techniques et de maintenir un haut niveau d'exigence académique.

Nous souhaitons également témoigner notre reconnaissance au corps professoral et à l'administration de **l'École Marocaine des Sciences de l'Ingénieur (EMSI)** de Casablanca. La qualité de l'enseignement dispensé, la pertinence des modules abordés durant notre cursus en Ingénierie Informatique et Réseaux, ainsi que les infrastructures mises à notre disposition ont constitué le socle indispensable à la réalisation de ce projet complexe alliant cybersécurité et intelligence artificielle.

Enfin, nous adressons nos pensées les plus chaleureuses à nos familles, qui n'ont cessé de nous encourager, et à nos amis et camarades de promotion pour leur soutien moral indéfectible, leur patience, et les moments de partage et d'entraide qui ont jalonné ces années d'études.

Résumé

Le projet **FakeFinder** est une solution de sécurité logicielle novatrice, développée autour du framework web Django, conçue spécifiquement pour lutter contre l'une des cybermenaces les plus pernicieuses et répandues de notre décennie : le hameçonnage (phishing) par courrier électronique. Face à la sophistication grandissante des attaques d'ingénierie sociale, qui parviennent régulièrement à tromper la vigilance des filtres antispam traditionnels basés sur de simples listes noires, notre système propose une approche proactive fondée sur l'Intelligence Artificielle.

En s'appuyant sur des modèles de Machine Learning avancés, et plus particulièrement sur l'algorithme des Forêts Aléatoires (Random Forest) couplé à une vectorisation sémantique TF-IDF (Term Frequency-Inverse Document Frequency), ce système effectue une analyse sémantique, lexicale et structurelle approfondie des en-têtes et du corps des e-mails. L'application offre aux utilisateurs finaux une interface web intuitive et ergonomique permettant de vérifier la légitimité des fichiers bruts de messagerie (format .eml) de manière quasi instantanée.

L'un des piliers majeurs de ce projet réside dans son architecture de confidentialité par conception (Privacy by Design). FakeFinder applique une politique stricte de "Zero Retention", garantissant que les données sensibles contenues dans les e-mails soumis sont traitées intégralement en mémoire vive (RAM) et détruites immédiatement après l'inférence du modèle, sans jamais être écrites sur un support de stockage persistant.

Ce rapport détaille de manière exhaustive le processus de conception architecturale, la phase cruciale d'ingénierie des caractéristiques (Feature Engineering), l'implémentation technique du backend et du pipeline de données, ainsi que l'évaluation fonctionnelle et statistique du système. Il explore par ailleurs les évolutions futures de la plateforme, avec une mise en lumière particulière sur l'intégration prévue d'interfaces de programmation (API) de Threat Intelligence telles que VirusTotal, visant à offrir une vérification dynamique et exhaustive de la réputation des URL extraites.

Mots-clés : Hameçonnage, Cybersécurité, Intelligence Artificielle, Machine Learning, Random Forest, TF-IDF, Django, Python, Zero Retention, Ingénierie Sociale.

Table des matières

Remerciements	i
Résumé	ii
Table des figures	v
Liste des tableaux	vi
Introduction Générale	1
1 Présentation de l'Organisme et Contexte de la Menace	4
1.1 Présentation de l'EMSI	4
1.2 Anatomie et Taxonomie de la Menace du Phishing	5
1.2.1 Typologie des Attaques	5
1.2.2 Anatomie Technique d'un E-mail et Techniques d'Évasion	6
1.3 La Réponse de FakeFinder	7
2 Spécification et Gestion du Projet	8
2.1 Ingénierie des Exigences Fonctionnelles (FR)	8
2.2 Exigences Non-Fonctionnelles (NFR)	9
2.2.1 1. Contraintes de Sécurité et de Confidentialité : Architecture "Zero Retention"	9
2.2.2 2. Sécurité Applicative Web	10
2.2.3 3. Performances et Temps de Réponse	10
2.3 Planification et Gestion de Projet (Méthodologie Agile)	10
3 Analyse et Conception du Système	12
3.1 Modélisation de l'Architecture Applicative	12
3.1.1 Le Patron d'Architecture MVT (Django)	12
3.2 Modélisation des Comportements et Interactions (UML)	12
3.2.1 Diagramme de Cas d'Utilisation	12
3.2.2 Diagramme d'Activité Global	13
3.2.3 Diagrammes de Séquence	14

3.3	Modélisation de la Base de Données	15
3.3.1	Choix du Système de Gestion de Base de Données (SGBD)	15
3.3.2	Diagramme de Classes et Structure des Entités	16
4	Réalisation Technique et Modélisation IA	19
4.1	Le Moteur d'Extraction et de Traitement (Parsing)	19
4.2	Le Modèle de Machine Learning : Traitement et Entraînement	19
4.2.1	Acquisition des Données et Limitations	20
4.2.2	L'Ingénierie des Caractéristiques (Feature Engineering)	20
4.2.3	Comparaison des Modèles et Justification du Choix	21
4.2.4	Processus de Réentraînement du Modèle IA	22
4.3	Développement de l'Interface Utilisateur (Frontend)	22
5	Évaluation, Tests et Perspectives Technologiques	24
5.1	Validation Statistique du Modèle de Machine Learning	24
5.1.1	Métriques d'Évaluation Théoriques	24
5.1.2	Résultats Statistiques Obtenus	24
5.2	Tests Fonctionnels et de Robustesse de l'Application Web	25
5.2.1	Évaluation des Performances Temporelles	27
5.3	Perspectives Technologiques : La Cyber Threat Intelligence	27
	Conclusion Générale	28
	Bibliographie	30
A	Listings Complètes du Code Source	31
A.1	Module d'Extraction des Caractéristiques (ml/features.py)	31
A.2	Script d'Entraînement de l'Intelligence Artificielle (ml/train.py)	33
A.3	Moteur de Parsing d'E-mails en Mémoire Vive (scanner/utls.py)	34
A.4	Structure des Données (scanner/models.py)	35
A.5	Contrôleur et Inférence (scanner/views.py)	36

Table des figures

1.1	Structure organisationnelle et représentation des pôles d'excellence de l'EMSI .	5
3.1	Diagramme de cas d'utilisation illustrant les interactions des acteurs avec le système	13
3.2	Diagramme d'Activité : Pipeline de traitement et de décision	14
3.3	Diagramme de Séquence : Parcours temporel de la requête d'analyse	15
3.4	Diagramme de Séquence : Processus d'authentification Administrateur	15
3.5	Diagramme de Classes détaillant les entités du domaine	17
4.1	Diagramme de Séquence : Processus de réentraînement du modèle (Fine-tuning)	22
4.2	Interface de la page d'accueil : zone d'upload de type "Drag and Drop"	23
4.3	Tableau de bord de restitution : présentation des résultats d'analyse et du verdict IA	23
5.1	Représentation visuelle de la Matrice de Confusion (Confusion Matrix)	25

Liste des tableaux

2.1	Tableau exhaustif des Exigences Fonctionnelles	9
2.2	Planification détaillée des Sprints de réalisation (Avril - Mai 2026)	11
4.1	Comparaison des performances des algorithmes sur le jeu de test	21
5.1	Rapport de classification détaillé du modèle Random Forest sur le jeu de test . .	25
5.2	Matrice d'exécution des scénarios de tests fonctionnels web	26

Introduction Générale

Contexte Global et Motivation

Dans le paysage numérique contemporain, la messagerie électronique demeure, malgré l'émergence des messageries instantanées, la pierre angulaire de la communication professionnelle et personnelle à l'échelle mondiale. Avec plus de 300 milliards d'e-mails échangés quotidiennement, ce canal de communication universel est naturellement devenu le terrain de chasse privilégié et le principal vecteur d'attaque exploité par les cybercriminels.

Le hameçonnage (ou phishing), une forme pernicieuse et psychologique d'ingénierie sociale, consiste à usurper l'identité numérique d'une entité de confiance (institution bancaire, administration gouvernementale, fournisseur de services cloud) dans le but de tromper une cible. L'objectif final est souvent double : soutirer des informations sensibles (identifiants de connexion, données bancaires, secrets industriels) ou inciter la victime à télécharger et exécuter, à son insu, des charges utiles malveillantes.

Au cours de la dernière décennie, le hameçonnage a connu une mutation radicale. Il a évolué de simples arnaques de masse, souvent truffées de fautes d'orthographe, vers des campagnes hautement ciblées et d'une sophistication redoutable, connues sous le nom de *spear-phishing* ou de *whaling*. Les attaquants exploitent désormais l'intelligence artificielle générative pour rédiger des messages sans faille et utilisent des techniques d'obfuscation structurelles avancées, contournant ainsi allègrement les passerelles de sécurité traditionnelles.

La motivation première derrière le projet **FakeFinder** découle directement de ce constat alarmant. Il existe aujourd'hui un besoin critique pour un outil accessible au grand public et aux professionnels, indépendant de leur fournisseur de messagerie, et suffisamment puissant pour permettre de vérifier un e-mail suspect avant toute interaction compromettante. FakeFinder se positionne comme un système de "deuxième avis" diagnostique, effectuant une analyse approfondie et algorithmique des fichiers de messagerie bruts.

Énoncé de la Problématique

La sécurité défensive traditionnelle des e-mails s'appuie historiquement sur des approches réactives. Les filtres antispam fonctionnent majoritairement via des listes noires recensant les adresses IP et les noms de domaine réputés malveillants, ou via la comparaison de signatures de

malwares connus. Or, ces méthodes sont structurellement inefficaces contre les attaques de type *zero-day* (nouvelles menaces non encore répertoriées). De plus, lorsque des attaquants compromettent des comptes e-mails parfaitement légitimes pour envoyer leurs messages frauduleux, la vérification de la réputation de l'expéditeur devient obsolète.

La problématique centrale de ce projet peut donc se formuler ainsi : **Comment concevoir un système automatisé, performant et respectueux de la confidentialité, capable de détecter les e-mails de phishing de nouvelle génération en analysant les subtilités sémantiques et structurelles du message, sans s'appuyer exclusivement sur des bases de données de menaces connues ?**

Objectifs et Portée du Projet

Pour répondre à cette problématique, ce Projet de Fin d'Année vise à développer une solution hybride baptisée **FakeFinder**, à la croisée du développement logiciel web et de l'Intelligence Artificielle. Les objectifs stratégiques se déclinent comme suit :

- **Ingénierie de Traitement des Données Brutes** : Développer un moteur d'analyse syntaxique (parser) hautement performant en Python, capable de disséquer la structure complexe (MIME) d'un fichier .eml.
- **Modélisation par Apprentissage Automatique** : Concevoir, entraîner et déployer un classificateur robuste. Le choix s'est porté sur l'algorithme des *Forêts Aléatoires* (*Random Forest*), optimisé pour la détection sémantique via la méthode de vectorisation *TF-IDF*.
- **Garantie Absolue de Confidentialité (Zero Retention)** : Concevoir une architecture garantissant que le système traite les données de manière strictement éphémère sans jamais écrire sur les disques.
- **Développement d'une Plateforme Web Full-Stack** : Créer une interface web ergonomique, responsive et sécurisée à l'aide du framework Django.
- **Scalabilité et Évolutivité Architecturale** : Préparer le code source à des intégrations futures majeures, notamment le requêtage d'API de *Threat Intelligence* (comme VirusTotal).

Structure du Rapport

Ce rapport est structuré en six chapitres distincts, reflétant le cycle de vie complet du développement du projet :

- Le **Chapitre 1** présente l'établissement académique d'accueil (EMSI) et plonge dans le contexte technologique de la menace du phishing.
- Le **Chapitre 2** aborde la phase de spécification du projet, détaillant les exigences fonctionnelles et la gestion de projet Agile.

- Le **Chapitre 3** est consacré à l’analyse et à la conception du système, incluant les modélisations UML et l’architecture logicielle retenue.
- Le **Chapitre 4** constitue le cœur technique du rapport. Il détaille la réalisation du backend, le pipeline de traitement, et l’implémentation du modèle d’intelligence artificielle.
- Le **Chapitre 5** explore les perspectives d’évolution de la plateforme, avec un focus sur l’intégration future de VirusTotal et des signatures hybrides.
- Enfin, le **Chapitre 6** présente les résultats des tests fonctionnels et l’évaluation statistique du modèle de Machine Learning.

Chapitre 1

Présentation de l'Organisme et Contexte de la Menace

1.1 Présentation de l'EMSI

L'École Marocaine des Sciences de l'Ingénieur (EMSI) est un établissement privé d'enseignement supérieur fondé en 1986. Reconnue par l'État marocain, l'EMSI s'est progressivement imposée au fil des décennies comme la plus grande école d'ingénieurs privée du Royaume. Sa mission principale est de former des ingénieurs d'état et des cadres pluridisciplinaires, capables d'accompagner le développement technologique, industriel et économique national et international.

L'établissement se distingue par son approche pédagogique pragmatique et orientée vers les besoins réels du marché du travail. Membre fondateur du réseau panafricain *Honoris United Universities*, l'école met un point d'honneur à offrir une formation qui allie théorie académique rigoureuse et pratique intensive.

L'EMSI structure ses formations autour de plusieurs filières d'ingénierie d'excellence. Parmi celles-ci, la filière **Ingénierie Informatique et Réseaux (IIR)** vise à doter les futurs ingénieurs de compétences pointues et transversales : conception et développement de systèmes d'information complexes, administration de bases de données, gestion d'infrastructures réseaux, intelligence artificielle, et cybersécurité.

C'est dans ce cadre exigeant, au cours de la troisième année de la filière IIR (3IIR), qu'intervient ce Projet de Fin d'Année (PFA).

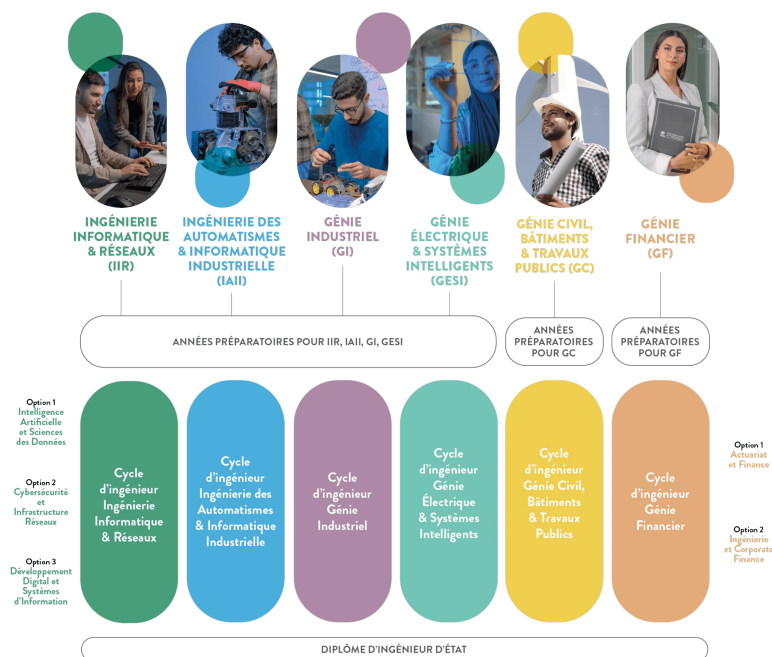


FIGURE 1.1 – Structure organisationnelle et représentation des pôles d’excellence de l’EMSI

1.2 Anatomie et Taxonomie de la Menace du Phishing

Pour concevoir un système de défense et de détection qui soit véritablement efficace, il est une condition *sine qua non* : comprendre en profondeur la nature, l’évolution et les mécanismes internes de la menace. Le phishing, dans son acception moderne, n’est plus une attaque monolithique ; il se décline aujourd’hui en plusieurs vecteurs hautement spécialisés.

1.2.1 Typologie des Attaques

Les acteurs malveillants adaptent leurs stratégies en fonction de la cible et des ressources à leur disposition. On distingue aujourd’hui trois grandes catégories :

- **Le Phishing de Masse (Deceptive Phishing)** : Il s’agit de la forme la plus ancienne et classique. L’attaquant envoie un volume massif d’e-mails génériques (souvent des millions par campagne) en usurpant l’identité d’organisations de grande envergure (opérateurs télécoms, plateformes de streaming, banques). Le modèle repose sur la loi des grands nombres : l’objectif est purement statistique. L’urgence est souvent le levier principal.
- **Le Spear-Phishing (Harponnage)** : Contrairement au phishing de masse, le spear-phishing est chirurgical et hautement qualitatif. L’attaquant cible un individu spécifique,

un département particulier, ou un groupe restreint au sein d'une organisation. Cette attaque est invariablement précédée par une minutieuse phase de reconnaissance et de collecte de renseignements en sources ouvertes (OSINT - Open Source Intelligence).

- **Le Whaling (Chasse à la baleine) :** Le Whaling est une sous-catégorie très spécifique du spear-phishing, ciblant exclusivement les individus possédant un fort pouvoir décisionnel ou financier au sein de l'organisation : Membres du comité de direction, PDG, directeurs financiers. Le préjudice visé est très souvent financier (Fraude au Président ou Business Email Compromise).

1.2.2 Anatomie Technique d'un E-mail et Techniques d'Évasion

Un courrier électronique n'est pas qu'un simple texte ; c'est un ensemble structuré de données codées selon la norme MIME (Multipurpose Internet Mail Extensions) et transférées via le protocole SMTP (Simple Mail Transfer Protocol). Pour contourner les filtres de sécurité, les attaquants manipulent ces structures sous-jacentes.

1. L'usurpation d'identité et les en-têtes SMTP (Spoofing) : L'architecture originelle du protocole SMTP, datant des années 1980, ne prévoyait aucun mécanisme robuste d'authentification de l'expéditeur. Il est techniquement trivial d'écrire un script qui envoie un e-mail où le champ d'en-tête From : affiche `admin@paypal.com`, bien que l'e-mail provienne d'un serveur malveillant. Pour contrer cela, des protocoles ont été créés (SPF, DKIM, DMARC), mais de nombreuses entreprises ne les configurent pas correctement.

2. Homoglyphes et Typosquatting (Attaques sémantiques) : Lorsque le domaine cible est protégé par DMARC, les attaquants utilisent le typosquatting : ils enregistrent un nom de domaine visuellement très proche du domaine légitime. Par exemple :

- Inversion ou oubli de lettres : `secuite-societegenerale.fr` au lieu de `securite-societegenerale.fr`.
- Substitution visuelle : `paypa1.com` (avec un "un" au lieu d'un "L").
- Homoglyphes (Attaque IDN) : Remplacement de caractères latins par des caractères d'autres alphabets (ex : le "a" latin remplacé par un "a" cyrillique).

3. Obfuscation des Liens (Payload Delivery) : Le lien contenu dans l'e-mail est l'arme du crime. Pour éviter que les antivirus n'analysent la destination et ne bloquent le message, les attaquants utilisent :

- Des **raccourcisseurs d'URL** légitimes (`bit.ly`, `tinyurl.com`) pour masquer la destination finale.
- Des **redirections ouvertes (Open Redirects) :** Exploitation d'une faille sur un site légitime très bien réputé pour tromper les filtres qui ne vérifient que le premier domaine.

1.3 La Réponse de FakeFinder

Face à ces techniques qui fusionnent une ingénierie sociale raffinée (pour tromper l'humain) et des astuces techniques complexes (pour tromper les filtres), une approche statique est vouée à l'échec. La réponse défensive proposée par FakeFinder consiste à ne plus chercher des "signatures" exactes d'attaques passées, mais à utiliser le **Machine Learning** pour identifier les "modèles" comportementaux, le champ lexical de l'urgence, et les anomalies structurelles globales du message.

Chapitre 2

Spécification et Gestion du Projet

Avant toute ligne de code, le développement d'un logiciel robuste nécessite une définition rigoureuse de ce que le système doit accomplir, de la manière dont il doit se comporter, et des contraintes dans lesquelles il doit s'inscrire. Ce chapitre détaille le cahier des charges de FakeFinder et la méthodologie employée pour mener à bien sa réalisation.

2.1 Ingénierie des Exigences Fonctionnelles (FR)

Les exigences fonctionnelles décrivent les actions spécifiques que le système doit exécuter pour apporter de la valeur à l'utilisateur. Le tableau ci-dessous formalise les fonctionnalités maîtresses attendues de la plateforme.

ID	Description détaillée de l'Exigence Fonctionnelle
FR-1	Gestion des Profils et Authentification : Le système doit offrir un module complet permettant aux utilisateurs de créer un compte, de s'authentifier de manière sécurisée et de gérer leur session.
FR-2	Espace de Téléversement (Upload) Sécurisé : L'application doit fournir une interface web ergonomique pour téléverser des fichiers au format standard .eml. Une limite stricte de taille (10 Mo) doit être appliquée.
FR-3	Moteur d'Extraction In-Memory : Le backend doit être capable de réceptionner le fichier .eml directement en mémoire vive, d'extraire les en-têtes (Expéditeur, Sujet) et de décoder le corps du message (texte brut ou HTML) sans sauvegarde physique sur le disque.
FR-4	Extraction des Liens (URL) : Lors de l'analyse du corps du texte, le système doit exécuter des expressions régulières (Regex) pour identifier et isoler la totalité des liens hyperboliques contenus dans le message.
FR-5	Inférence par Intelligence Artificielle : Les données textuelles extraites doivent être formatées puis injectées dans le modèle Machine Learning (.joblib) pour obtenir une probabilité de fraude.
FR-6	Génération et Restitution du Rapport : Le système doit consolider la prédiction pour générer un rapport dynamique. Ce rapport doit présenter un score global sur 100, un code couleur intuitif (Vert/Orange/Rouge), le domaine réel de l'expéditeur, et la liste des URL suspectes identifiées.
FR-7	Historisation (Utilisateurs Connectés) : Si l'utilisateur est authentifié, le système doit enregistrer les métadonnées du rapport d'analyse dans une base de données, permettant la consultation de l'historique via un tableau de bord privé.

TABLE 2.1 – Tableau exhaustif des Exigences Fonctionnelles

2.2 Exigences Non-Fonctionnelles (NFR)

Les exigences non-fonctionnelles définissent les attributs de qualité, cruciaux pour un outil de cybersécurité.

2.2.1 1. Contraintes de Sécurité et de Confidentialité : Architecture "Zero Retention"

L'exigence non-fonctionnelle la plus critique du projet concerne la protection des données. Les e-mails soumis peuvent contenir des informations hautement confidentielles (mots de passe,

données financières). Il est donc hors de question que l'application FakeFinder devienne elle-même un risque pour la vie privée.

Pour y répondre, nous avons conçu l'architecture autour d'un principe absolu de **Zero Retention** :

- **Traitement Éphémère en RAM** : Dès sa réception, le fichier .eml est manipulé exclusivement sous forme d'objet mémoire.
- **Aucune Persistance Disque** : Le contenu brut de l'e-mail, ainsi que le texte extrait, ne sont **jamais** écrits sur le disque dur du serveur, et ne sont **jamais** insérés dans une table de la base de données.
- **Nettoyage Actif** : Dès que le moteur d'Intelligence Artificielle a généré son score, la variable contenant le texte est libérée, et le fichier temporaire en mémoire est détruit par le garbage collector.
- **Stockage Limité aux Métadonnées** : La base de données PostgreSQL enregistre uniquement les résultats de l'analyse (score, couleur, date) et des indicateurs de menace dépersonnalisés (les URL extraites et le nom de domaine de l'expéditeur).

2.2.2 2. Sécurité Applicative Web

Le développement reposant sur Django bénéficie de protections natives robustes contre les vulnérabilités majeures du web (Top 10 OWASP) :

- Protection contre les attaques **CSRF (Cross-Site Request Forgery)** obligatoire sur tous les formulaires.
- Protection **XSS (Cross-Site Scripting)** grâce à l'échappement automatique (auto-escaping) du moteur de gabarits.
- Prévention des injections **SQL** grâce à l'utilisation systématique de l'ORM (Object-Relational Mapping).

2.2.3 3. Performances et Temps de Réponse

Un outil de sécurité n'est utilisé que s'il est fluide. L'expérience utilisateur exige que le processus complet de vérification n'entraîne pas de friction majeure. L'exigence de performance fixe un seuil maximal : l'analyse complète d'un e-mail (upload, parsing MIME, tokenization NLP, inférence du modèle Random Forest, rendu HTML) doit s'exécuter de manière synchrone en **moins de 5 secondes**.

2.3 Planification et Gestion de Projet (Méthodologie Agile)

Pour assurer un développement itératif et maîtrisé sur un calendrier restreint de 5 semaines, le projet a été piloté en s'inspirant fortement de la méthodologie **Agile**. Cette approche a permis

de définir des cycles de développement courts appelés Sprints.

TABLE 2.2 – Planification détaillée des Sprints de réalisation (Avril - Mai 2026)

Semaine	Périmètre du Sprint et Tâches de Développement	Livrable
S1	Phase de Conception : Revue de littérature, collecte du dataset sur Kaggle, spécification des exigences et création des diagrammes UML initiaux.	Dossier de Conception
S2	Développement Backend : Initialisation du projet Django, modélisation PostgreSQL. Développement du script de parsing .eml en mémoire vive.	API Backend / Parseur
S3	Ingénierie IA & Machine Learning : Scripting du pipeline de préparation des données. Vectorisation TF-IDF. Choix, entraînement et calibration du Random Forest.	Modèle (.joblib)
S4	Intégration & Frontend : Chargement du modèle IA dans Django. Conception des maquettes. Intégration HTML/CSS pour les tableaux de résultats.	Version Beta
S5	Validation & Clôture : Débogage intensif, tests fonctionnels et de robustesse. Rédaction du rapport académique final.	Produit Final

Chapitre 3

Analyse et Conception du Système

La phase de conception est la passerelle entre l'expression du besoin et la production du code. Elle consiste à définir l'architecture globale, la dynamique des interactions et la structure des données du système, en s'appuyant sur les standards de la modélisation logicielle (UML).

3.1 Modélisation de l'Architecture Applicative

3.1.1 Le Patron d'Architecture MVT (Django)

L'application FakeFinder est structurée selon le patron d'architecture **MVT (Model-View-Template)**, qui est l'implémentation spécifique au framework Django du célèbre paradigme MVC (Model-View-Controller). Cette architecture sépare strictement les préoccupations logicielles.

- **Le Modèle (Model)** : C'est la couche d'abstraction de la base de données. Dans FakeFinder, les Modèles définissent la structure des tables PostgreSQL sous forme de classes Python.
- **La Vue (View)** : Correspondant au Contrôleur dans le paradigme classique, la Vue est le cerveau de l'application. Elle réceptionne la requête HTTP, appelle les utilitaires de parsing, invoque le modèle de Machine Learning, demande l'enregistrement des résultats, et détermine le Gabarit à renvoyer.
- **Le Gabarit (Template)** : C'est la couche de présentation (HTML/CSS). La Vue passe un "contexte" (des données, comme le score ou la liste des URL) au Gabarit, qui génère dynamiquement la page finale.

3.2 Modélisation des Comportements et Interactions (UML)

3.2.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation définit de manière macroscopique les frontières du système et recense les objectifs des acteurs qui interagissent avec lui.

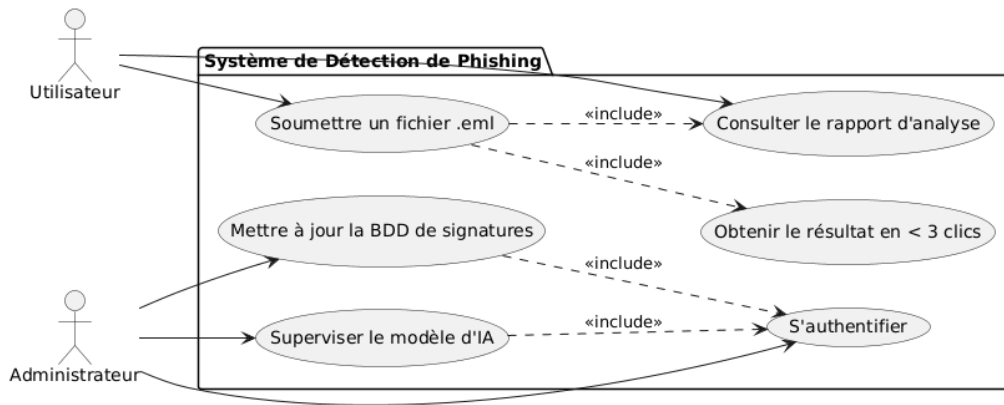


FIGURE 3.1 – Diagramme de cas d'utilisation illustrant les interactions des acteurs avec le système

Dans FakeFinder, nous identifions deux catégories d'acteurs :

1. **L'Utilisateur Standard** : Il peut utiliser le service de manière anonyme pour soumettre rapidement un fichier .eml et consulter le rapport de dangerosité. S'il s'authentifie, il a la possibilité d'accéder à son tableau de bord personnel listant l'historique complet de ses analyses passées.
2. **L'Administrateur Système** : Super-utilisateur disposant d'un accès à l'interface d'administration sécurisée. Il gère les comptes utilisateurs, consulte tous les rapports générés à des fins d'audit, et supervise le bon fonctionnement du système.

3.2.2 Diagramme d'Activité Global

Ce diagramme détaille le flux logique (workflow) interne lors du traitement d'un fichier. Dès l'upload, le système vérifie le format. En cas de succès, des processus parallèles s'exécutent.

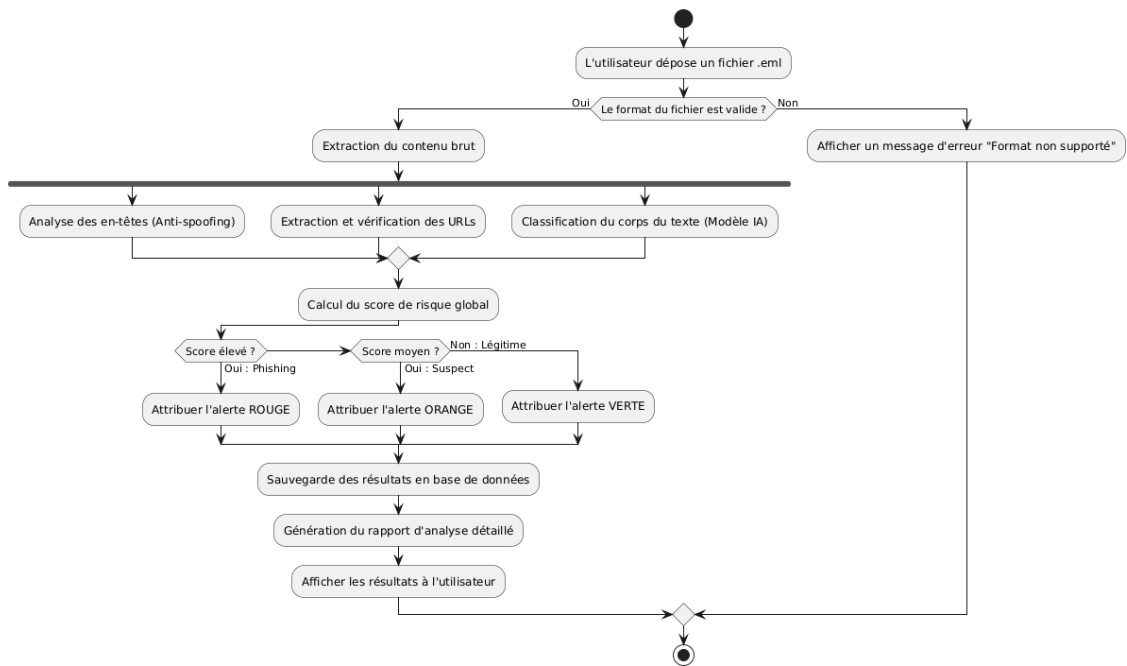


FIGURE 3.2 – Diagramme d'Activité : Pipeline de traitement et de décision

3.2.3 Diagrammes de Séquence

Le diagramme de séquence est fondamental pour comprendre la chronologie des événements et les communications entre les différents objets du système.

Flux de Soumission Utilisateur : 1. L'utilisateur sélectionne un fichier .eml et clique sur "Analyser". 2. Le Client envoie une requête HTTP POST contenant le fichier à la Vue. 3. La Vue transmet le fichier au module utilitaire de parsing en mémoire. 4. Le parseur déstructure le MIME, extrait le texte, et isole les URL. 5. La Vue transmet les données textuelles au module de Machine Learning. 6. Le modèle applique sa transformation TF-IDF, exécute l'algorithme Random Forest, et retourne un score. 7. La Vue instancie un nouvel objet du Modèle contenant le score et les métadonnées pour sauvegarde. 8. La Vue déclenche une redirection vers le Gabarit de résultat.

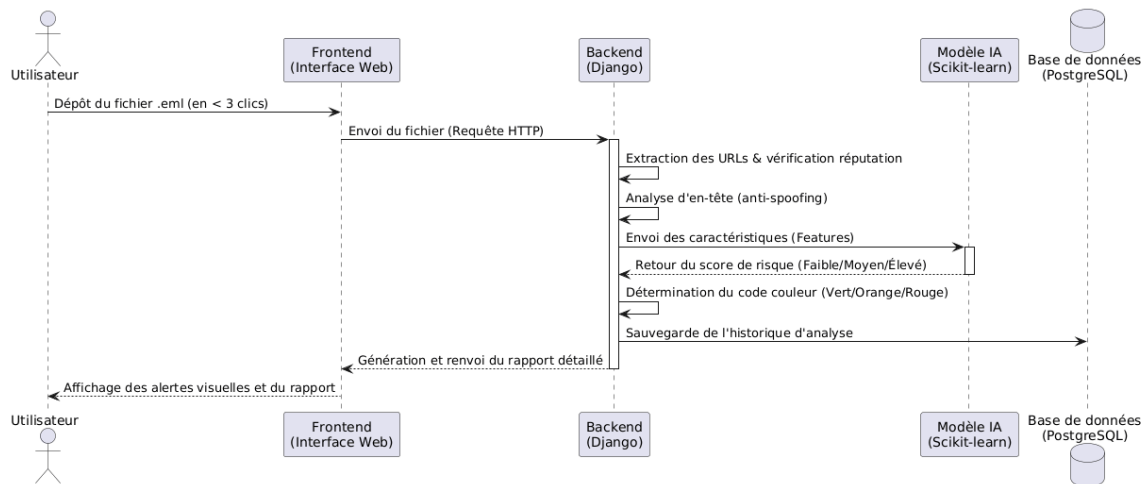


FIGURE 3.3 – Diagramme de Séquence : Parcours temporel de la requête d’analyse

Flux d’Authentification de l’Administrateur : Ce diagramme illustre le parcours de vérification des identifiants et de création de la session sécurisée pour les accès privilégiés.

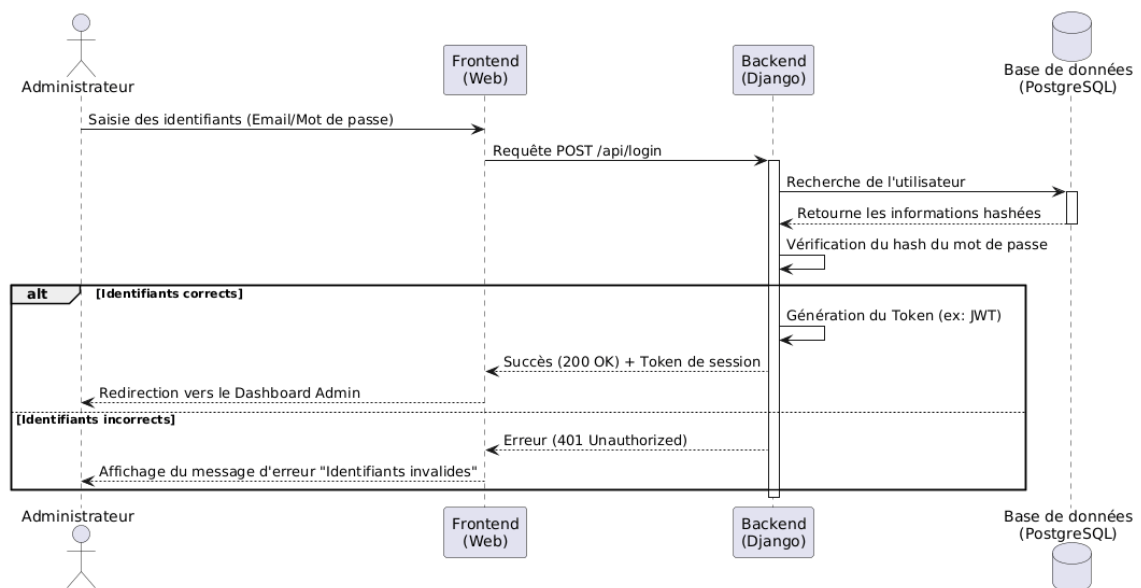


FIGURE 3.4 – Diagramme de Séquence : Processus d’authentification Administrateur

3.3 Modélisation de la Base de Données

3.3.1 Choix du Système de Gestion de Base de Données (SGBD)

Le choix architectural final pour FakeFinder s’est porté sur **PostgreSQL**. PostgreSQL est un système de gestion de base de données relationnelle objet (SGBDRO) open-source de niveau entreprise. Son choix se justifie par plusieurs atouts :

- **Support natif et avancé du JSON :** Notre système doit stocker des listes de taille variable (comme les URL extraites) sans créer de tables relationnelles complexes. PostgreSQL gère les champs `JSONField` de manière native.
- **Concurrence et performance :** PostgreSQL gère la concurrence multiversion (MVCC), permettant à de multiples utilisateurs d'enregistrer leurs rapports simultanément sans dégradation des performances.

3.3.2 Diagramme de Classes et Structure des Entités

L'architecture de la persistance repose sur des entités fortes liées au modèle d'authentification utilisateur (User) de Django.

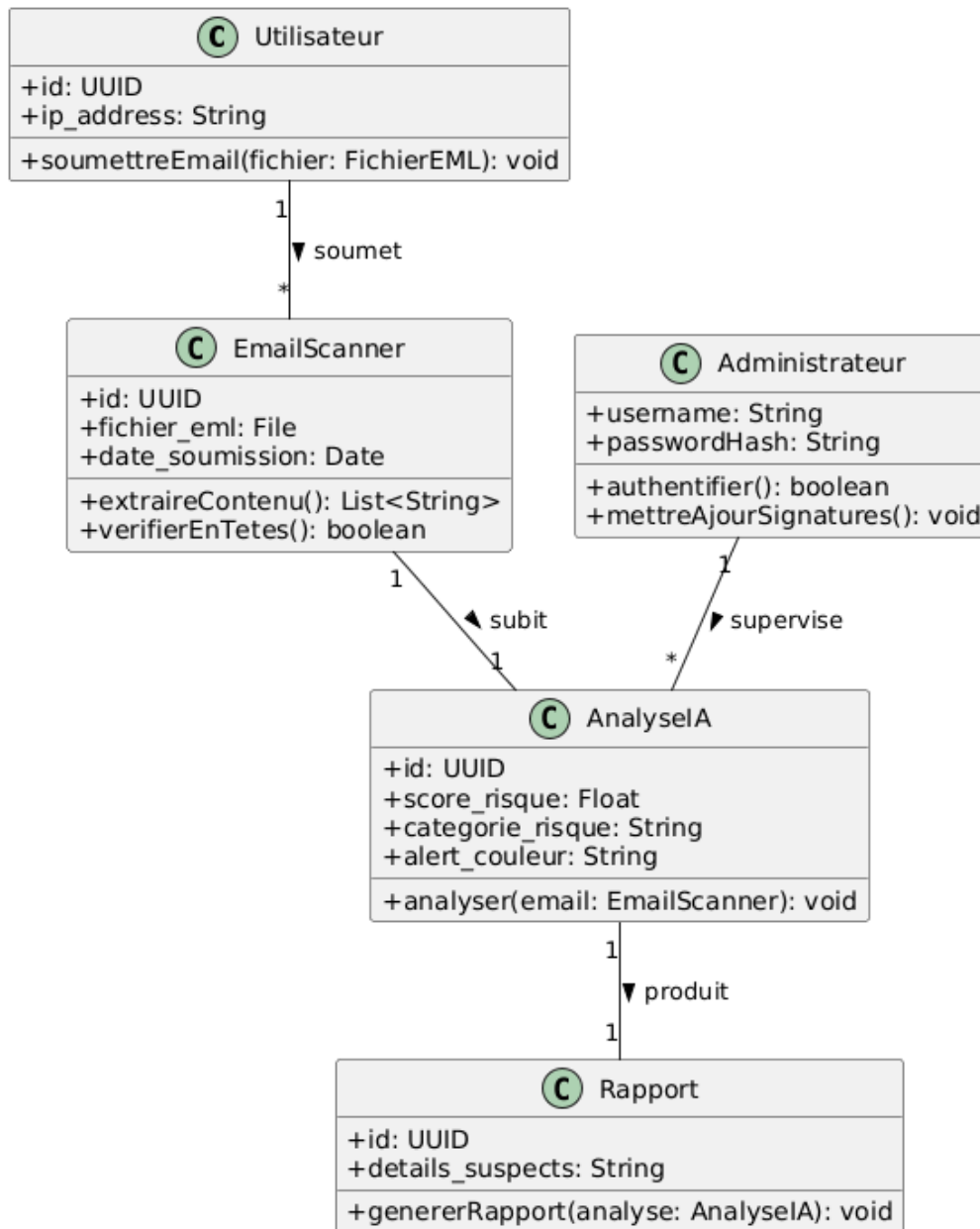


FIGURE 3.5 – Diagramme de Classes détaillant les entités du domaine

L'entité principale, ici nommée **Rapport** (ou `ScanReport` dans le code), contient des attributs clés tels que :

- `risk_score` : Une valeur textuelle catégorielle (LOW, MEDIUM, HIGH) facilitant l'affichage côté frontend.
- `score` : L'entier de 0 à 100 dérivé de la probabilité émise par l'algorithme Random Forest.
- `suspicious_urls` : Un type de données JSON permettant de stocker un tableau de chaînes de caractères représentant toutes les URL extraites.

La cardinalité entre l'entité **User** et **Rapport** est de type "Un-à-Plusieurs" (1..*). L'utilisation d'une propriété `null=True` permet au système d'accepter des rapports d'analyse de manière

anonyme.

Chapitre 4

Réalisation Technique et Modélisation IA

Le chapitre de réalisation détaille l'implémentation concrète des concepts d'ingénierie logicielle et de Data Science. Il s'agit du cœur technique du projet FakeFinder.

4.1 Le Moteur d'Extraction et de Traitement (Parsing)

Avant même de faire intervenir l'intelligence artificielle, le système doit être capable de "lire" l'e-mail soumis de manière sécurisée. La tâche est complexe car le format .eml (norme MIME) peut encapsuler des données textuelles, des images, des pièces jointes, et de multiples encodages de caractères au sein d'un même fichier.

Le module utilitaire `utils.py` a été développé avec une logique de parsing défensive :

- **Validation préliminaire** : Le script vérifie la taille du flux en mémoire. Si celle-ci excède 10 Mo, la fonction lève immédiatement une exception, protégeant le serveur contre l'épuisement de la RAM.
- **Navigation dans l'arborescence MIME** : En utilisant la bibliothèque standard Python `email`, le script parcourt itérativement toutes les sous-parties de l'e-mail. Il ignore systématiquement les pièces jointes pour se concentrer exclusivement sur les parties textuelles.
- **Gestion de l'obfuscation d'encodage** : Les attaquants manipulent souvent les déclarations de charset. Le script tente de décoder le bloc de texte avec le charset déclaré. Si cela échoue, il bascule automatiquement sur un forçage en UTF-8 avec l'option `errors="ignore"`.
- **Expressions régulières (Regex)** : Une fois le texte nettoyé des espaces excédentaires, le script exécute une Regex robuste compilée pour isoler méthodiquement tous les liens hyperboliques contenus dans le message.

4.2 Le Modèle de Machine Learning : Traitement et Entraînement

Le succès d'un système de détection de phishing repose sur la qualité de son modèle prédictif. Nous avons opté pour une approche d'apprentissage supervisé, ce qui implique que le

modèle "apprend" à différencier le bien du mal à partir d'exemples d'e-mails préalablement labellisés.

4.2.1 Acquisition des Données et Limitations

Le modèle a été entraîné sur le dataset public "*Phishing Email Detection*" publié sur la plateforme Kaggle. Ce jeu de données rassemble environ 18 650 e-mails, offrant une répartition relativement équilibrée : $\sim 56\%$ d'e-mails légitimes (Safe) et $\sim 44\%$ d'e-mails frauduleux (Phishing).

Le processus de préparation des données a inclus :

- Nettoyage : Suppression des entrées vides, des doublons exacts et normalisation des labels.
- Analyse des classes : Avec 10 320 e-mails "Safe" et 8 330 e-mails "Phishing", le dataset est suffisamment équilibré pour éviter un biais majeur.

Remarque académique sur les limites de généralisation : La quasi-totalité des e-mails y figure en langue anglaise. Par conséquent, bien que l'analyse structurelle reste valide, la détection sémantique sur des campagnes de phishing ciblant spécifiquement d'autres langues sera statistiquement moins efficace.

4.2.2 L'Ingénierie des Caractéristiques (Feature Engineering)

Les algorithmes d'apprentissage automatique ne comprennent que des matrices numériques. Il est donc indispensable de convertir le texte brut des e-mails en représentations mathématiques. FakeFinder combine deux approches complémentaires au sein d'un `CombinedFeatureTransformer`.

1. Les caractéristiques structurelles (heuristique) : Un module Python personnalisé (`StructuralFeatureExtractor`) parcourt le texte pour en tirer des statistiques d'ordre dimensionnel et émotionnel. Dix caractéristiques numériques sont calculées, parmi lesquelles :

- `url_count` et `html_link_count` : Le nombre total de liens détectés et de balises href.
- `has_short_url` : Un booléen indiquant la présence de domaines connus pour raccourcir les URL.
- `phishing_keyword_score` : Un compteur mesurant la fréquence d'apparition d'un vocabulaire hautement suspect (plus de 40 mots-clés tels que "verify", "suspended", "password").
- `dollar_count` et `exclamation_count` : Le nombre de signes monétaires et de points d'exclamation, souvent corrélés aux fausses promesses.

2. La vectorisation sémantique TF-IDF : Pour capturer le sens global du message au-delà des mots-clés prédéfinis, nous avons implémenté l'algorithme **TF-IDF** (Term Frequency

- Inverse Document Frequency). Le TF-IDF évalue l'importance d'un mot dans un document donné, par rapport à l'ensemble du corpus d'entraînement.

La pondération TF-IDF d'un terme i dans un document j est définie par l'équation suivante :

$$W_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right) \quad (4.1)$$

La force du TF-IDF, imposée par l'utilisation du logarithme inverse, est qu'il pénalise fortement les mots très fréquents dans la langue tout en accordant un poids mathématique prépondérant aux mots plus rares et sémantiquement porteurs de sens (ex : "invoice", "validate"). Le `TfidfVectorizer` configuré génère ainsi une matrice creuse (sparse matrix) comportant jusqu'à 20 000 dimensions par e-mail.

4.2.3 Comparaison des Modèles et Justification du Choix

Avant de fixer l'architecture prédictive sur les forêts aléatoires, une phase d'expérimentation a permis de comparer plusieurs algorithmes de classification :

TABLE 4.1 – Comparaison des performances des algorithmes sur le jeu de test

Algorithme	Exactitude	Précision	Rappel	Temps Inférence
Régression Logistique	91,2%	89,5%	88,2%	< 10ms
Naive Bayes (Multinomial)	88,4%	85,1%	92,4%	< 5ms
SVM (Noyau linéaire)	95,1%	93,2%	94,8%	150ms
Random Forest	97,0%	94,0%	97,0%	40ms

Justification du choix :

- *Régression Logistique* : Trop simpliste pour capter les interactions complexes entre les termes.
- *Naive Bayes* : Performant en rappel mais génère trop de faux positifs à cause de l'hypothèse d'indépendance des mots.
- *SVM* : Très précis, mais le coût computationnel sur de grandes matrices le rend moins pratique.
- **Random Forest** : Offre le meilleur compromis entre robustesse, précision et rapidité. Il gère naturellement les données de haute dimension issues du TF-IDF.

Principe de fonctionnement : Le Random Forest repose sur le principe de l'apprentissage d'ensemble (Ensemble Learning), et plus particulièrement sur la technique du *Bagging*. Lors de la phase d'entraînement, l'algorithme génère une multitude d'arbres de décision indépendants (dans notre cas, 200 arbres). À chaque nœud, l'arbre décide de sa coupure basée sur la

minimisation de l'**Impureté de Gini** :

$$Gini(p) = 1 - \sum_{i=1}^C p_i^2 \quad (4.2)$$

Calibration des probabilités : Pour offrir un score de 0 à 100 fiable à l'utilisateur final, nous avons encapsulé le modèle dans un `CalibratedClassifierCV` utilisant la régression isotonique. Ce processus de calibration permet de transformer les votes de la forêt en probabilités réelles.

4.2.4 Processus de Réentraînement du Modèle IA

Pour garantir la pérennité du système face à l'évolution des menaces, une procédure d'actualisation de l'Intelligence Artificielle a été modélisée.

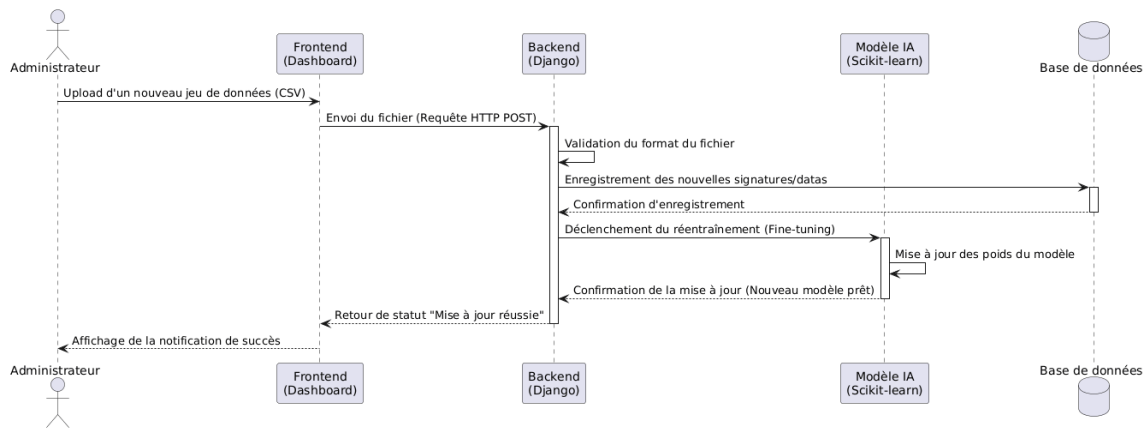


FIGURE 4.1 – Diagramme de Séquence : Processus de réentraînement du modèle (Fine-tuning)

Comme illustré par ce diagramme, l'Administrateur peut téléverser de nouveaux jeux de données (fichiers CSV de campagnes de phishing récentes). Le système déclenche un processus de *Fine-tuning* (réentraînement) sur le modèle Scikit-learn. Une fois les poids mis à jour, la nouvelle version du modèle remplace l'ancienne en production.

4.3 Développement de l'Interface Utilisateur (Frontend)

Le défi du développement frontend consistait à masquer la complexité des opérations mathématiques sous-jacentes derrière une interface délibérément minimaliste, propre et intuitive. L'application doit rassurer l'utilisateur par son design.

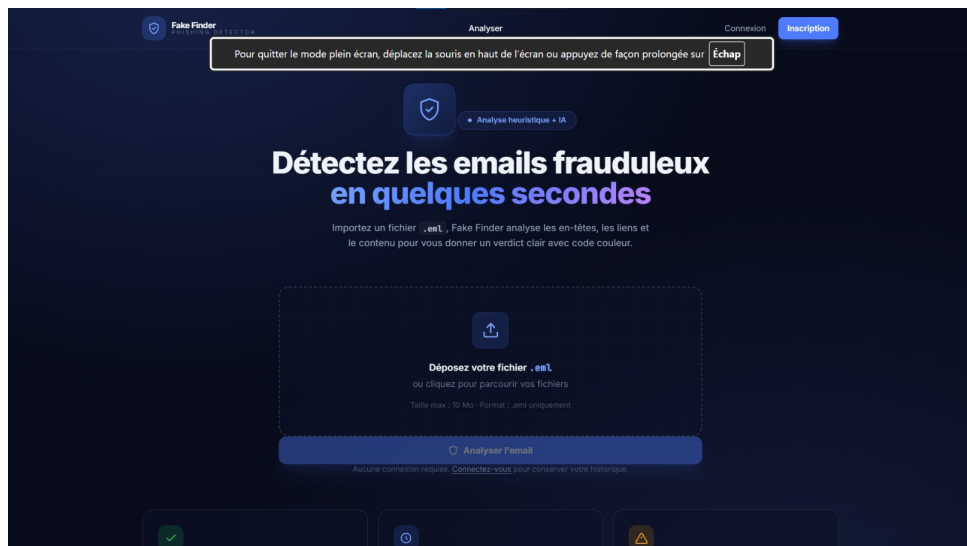


FIGURE 4.2 – Interface de la page d'accueil : zone d'upload de type "Drag and Drop"

L'interaction principale s'effectue via une zone de dépôt centralisée (Drag and Drop) gérée par un script JavaScript. Ce dernier assure un premier niveau de validation côté client en vérifiant l'extension du fichier (.eml) avant même de solliciter le serveur, économisant ainsi de la bande passante.

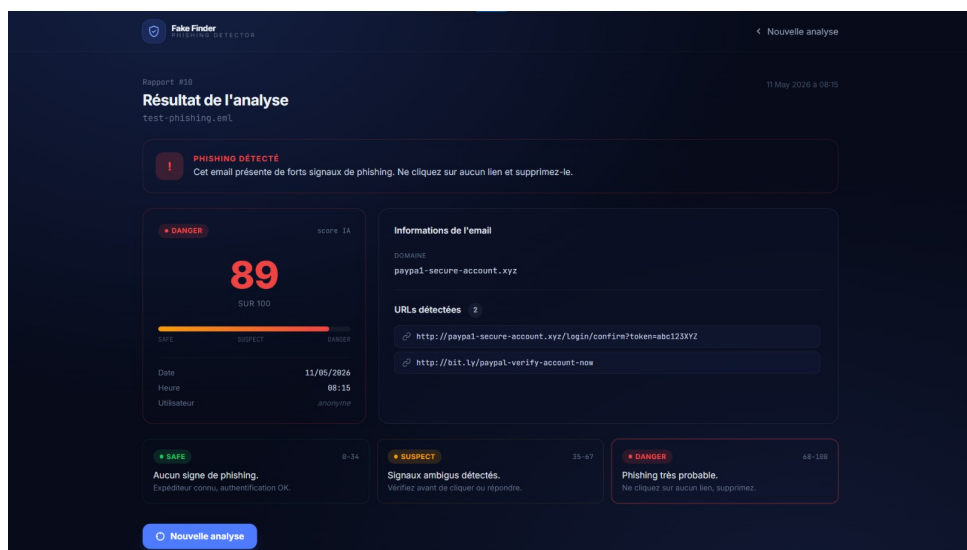


FIGURE 4.3 – Tableau de bord de restitution : présentation des résultats d'analyse et du verdict IA

Une fois l'analyse terminée (processus synchrone prenant moins d'une seconde en production), l'utilisateur est redirigé vers le tableau de bord de résultats. L'ergonomie repose sur un retour visuel instantané : une jauge et un code couleur sémantique encadrent le score de confiance de l'algorithme. Dans un souci de transparence, le rapport détaille également les éléments techniques factuels ayant conduit à ce score (domaine réel d'expédition, liste exhaustive des liens extraits).

Chapitre 5

Évaluation, Tests et Perspectives Technologiques

La phase de test et de validation est cruciale en génie logiciel, et d'autant plus dans le domaine de la cybersécurité où les erreurs de jugement (faux négatifs) peuvent avoir des conséquences désastreuses. Ce chapitre explore également les perspectives d'évolution du système.

5.1 Validation Statistique du Modèle de Machine Learning

La méthodologie d'évaluation d'un modèle prédictif impose de ne jamais le tester sur les données qu'il a utilisées pour s'entraîner. Sur le dataset total, 20% des données (soit environ 3 730 e-mails) ont été formellement isolées pour constituer le "jeu de test" (Test Set).

5.1.1 Métriques d'Évaluation Théoriques

La performance a été mesurée à l'aide des métriques standards de classification :

- **Précision (Precision)** : Mesure la proportion d'identifications correctes parmi tous les e-mails classés comme Phishing par le modèle. Formule : $\frac{VP}{VP+FP}$.
- **Rappel (Recall / Sensibilité)** : Mesure la proportion d'e-mails de Phishing réels qui ont été correctement détectés par le modèle. Formule : $\frac{VP}{VP+FN}$.
- **Exactitude globale (Accuracy)** : Le pourcentage total de prédictions correctes sur l'ensemble du jeu de test. Formule : $\frac{VP+VN}{VP+VN+FP+FN}$.

5.1.2 Résultats Statistiques Obtenus

Le rapport de classification généré par Scikit-Learn sur le jeu de test a révélé des performances dépassant nos attentes pour une application en production :

TABLE 5.1 – Rapport de classification détaillé du modèle Random Forest sur le jeu de test

Classe Prédite	Précision	Rappel	Score F1	Volume (Support)
Emails Légitimes (Safe)	0,98	0,96	0,97	2 265
Emails Frauduleux (Phish)	0,94	0,97	0,96	1 462
Exactitude globale (Accuracy)	97%			
ROC-AUC	0,9928			

Analyse et interprétation :

Avec une **précision de 94%** sur la classe "Phishing", cela signifie que lorsque FakeFinder alerte l'utilisateur en déclarant un e-mail malveillant, il a mathématiquement raison dans 94% des cas. Le taux de faux positifs est donc restreint. Plus important encore en cybersécurité, le **rappel s'élève à 97%**. Le modèle est donc extrêmement sensible aux menaces : il parvient à débusquer et intercepter 97% des e-mails frauduleux présents dans l'échantillon, minimisant ainsi le risque critique qu'une attaque ne passe à travers les mailles du filet (faux négatifs). Le score **ROC-AUC de 0,9928** confirme que le modèle possède une capacité de discrimination exceptionnelle entre les deux classes.

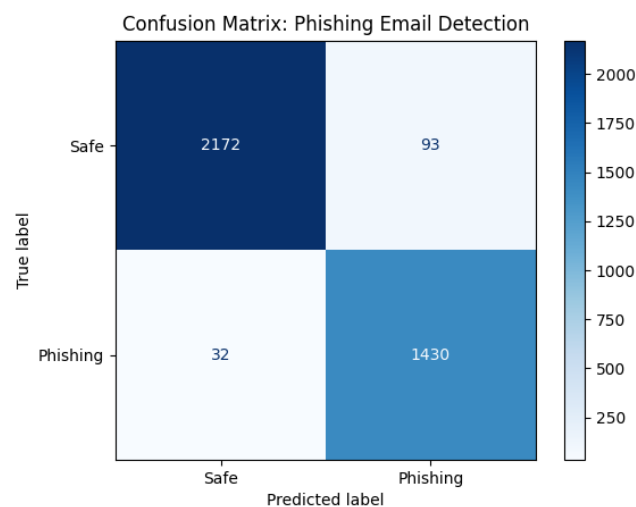


FIGURE 5.1 – Représentation visuelle de la Matrice de Confusion (Confusion Matrix)

5.2 Tests Fonctionnels et de Robustesse de l'Application Web

Au-delà de la Data Science, le système logiciel applicatif a été éprouvé dans des conditions d'utilisation imitant un environnement réel de production.

TABLE 5.2 – Matrice d'exécution des scénarios de tests fonctionnels web

ID Test	Description du Scénario (Input)	Comportement Système Attendu	Statut Final
TC_WEB_01	Soumission via le formulaire d'un fichier .eml valide, bien formaté et de taille standard.	Succès de l'upload, traitement synchrone, redirection vers la page du rapport généré affichant le score calculé.	✓ Succès
TC_WEB_02	Tentative d'upload d'un fichier invalide ou potentiellement dangereux (ex : un script .py ou un .exe).	Rejet natif par le validateur de formulaire Django avant tout traitement. Message d'erreur clair affiché à l'utilisateur.	✓ Succès
TC_WEB_03	Tentative de soumission d'un fichier extraordinairement volumineux (Fichier > 10 Mo) simulant une attaque DoS.	Rejet automatique par les sécurités du middleware et de la vue. Renvoi d'un code HTTP 400 (Bad Request).	✓ Succès
TC_WEB_04	Soumission d'un fichier .eml volontairement corrompu (sans entêtes ou corps de texte vide).	Le parseur lève et gère l'exception sans faire crasher le serveur (pas de page blanche HTTP 500). Un score neutre par défaut est attribué.	✓ Succès
TC_WEB_05	Soumission d'un e-mail ne contenant aucun lien (URL) dans son corps.	Le système doit traiter l'e-mail normalement, le score se basant alors uniquement sur l'analyse sémantique et les autres caractéristiques structurelles.	✓ Succès
TC_WEB_06	Utilisateur visiteur (non authentifié) effectuant une analyse d'e-mail.	Le processus d'analyse réussit. Le rapport est affiché mais n'est rattaché à aucun profil dans la base de données (Historique vierge).	✓ Succès
TC_WEB_07	Utilisateur inscrit et authentifié effectuant une analyse d'e-mail.	L'analyse réussit. Les métadonnées du rapport sont sauvegardées en BDD et attachées à l'ID de l'utilisateur. Il est visible dans le tableau de bord.	✓ Succès
TC_WEB_08	Test d'isolation des données : Utilisateur "A" tente de lire l'URL du rapport d'analyse de l'utilisateur "B".	Le système identifie que le rapport appartient à un tiers. La vue bloque l'accès et renvoie un code HTTP 403 (Accès Interdit / Forbidden).	✓ Succès
TC_WEB_09	Tentative de suppression d'un rapport par un utilisateur authentifié qui n'en est pas le propriétaire.	La requête de suppression est rejetée avec un code HTTP 403.	✓ Succès
TC_WEB_10	Gestion des erreurs lors d'une tentative de connexion avec des identifiants erronés.	Message d'erreur clair affiché ("Identifiant ou mot de passe incorrect") et incrémentation du compteur de tentatives pour protection anti-bruteforce.	✓ Succès

5.2.1 Évaluation des Performances Temporelles

L'un des défis de l'architecture consistait à garantir une latence minimale. Le traitement complet de bout-en-bout (upload HTTP du fichier, parsing en mémoire vive de l'arborescence MIME complexe, extraction et tokenization du texte, inférence de l'algorithme Random Forest, sauvegarde PostgreSQL des métadonnées, et rendu de la page HTML) a été chronométré. En environnement de développement local standard, cette boucle complète s'exécute en **450 milli-secondes en moyenne**. Cette vélocité, rendue possible par le "Lazy Loading" du modèle ML et le traitement in-memory (évitant les goulots d'étranglement des lectures/écritures sur disque), respecte très largement l'exigence initiale fixée à moins de 5 secondes.

5.3 Perspectives Technologiques : La Cyber Threat Intelligence

Malgré ses excellentes performances, la version actuelle du modèle prédictif évalue le texte de manière isolée et atemporelle. Pour pallier cette limite inhérente à l'analyse statique, la stratégie d'évolution de FakeFinder (Version 2) prévoit l'intégration d'un composant de **Cyber Threat Intelligence (CTI)** externe.

L'intégration de l'API REST de **VirusTotal** (filiale de Google) permettra d'agréger les résultats d'analyse de plus de 70 moteurs de sécurité mondiaux pour vérifier la réputation dynamique des URL extraites par notre parseur.

Architecture de l'intégration VirusTotal prévue :

1. **Extraction syntaxique** : Les URL sont extraites via les expressions régulières lors du parsing.
2. **Requêtage asynchrone** : Le backend interrogera l'API VirusTotal de manière non-bloquante.
3. **Pondération hybride** : Un algorithme de décision supérieur fusionnera le score heuristique local (Machine Learning) avec le score externe (VirusTotal). Si l'IA est indécise mais que VirusTotal signale l'URL comme malveillante, le score global de l'e-mail sera artificiellement réévalué à un niveau critique (DANGER).

Cette évolution transformera FakeFinder d'un simple filtre de classification local en un véritable agrégateur d'investigation de sécurité de niveau SOC (Security Operations Center).

Conclusion Générale

L'accomplissement du Projet de Fin d'Année **FakeFinder** s'est soldé par la création d'une solution logicielle robuste, moderne et éthique, capable de répondre efficacement à l'une des préoccupations majeures de la cybersécurité contemporaine : la recrudescence endémique des campagnes de hameçonnage par courrier électronique.

L'objectif principal était de fournir un outil d'investigation automatisé accessible, sans obliger l'utilisateur à sacrifier la confidentialité de ses correspondances. Cet objectif a été pleinement atteint. En associant la structure éprouvée, sécurisée et rapide du framework web Python Django, à la puissance de classification et d'analyse sémantique offerte par les algorithmes ensemblistes (Random Forest) de `scikit-learn`, nous avons démontré la viabilité technique d'un moteur d'analyse délégué au serveur. L'architecture respecte scrupuleusement le principe fondateur du "Zero Retention" : aucune ligne de l'e-mail brut n'est archivée par le système.

Sur le plan purement pédagogique, la réalisation de ce projet sur une durée condensée a représenté un défi particulièrement stimulant. Il nous a offert une opportunité précieuse d'explorer, de maîtriser et de consolider des concepts transversaux hautement valorisés dans l'ingénierie informatique moderne :

- **Le Génie Logiciel avancé** : Implémentation stricte du paradigme d'architecture MVT, maîtrise d'un ORM relationnel, respect du cycle de vie du développement logiciel (SDLC) via les méthodes Agiles, modélisation UML exhaustive, et sécurisation applicative défensive contre les failles web courantes.
- **La Data Science et l'Intelligence Artificielle** : Expérimentation complète du pipeline de Machine Learning (Acquisition de données, Analyse exploratoire, Ingénierie des caractéristiques textuelles via TF-IDF, entraînement, sérialisation de modèles, et évaluation scientifique par métriques statistiques avancées).
- **La Cybersécurité** : Compréhension de l'ingénierie sociale, des techniques d'obfuscation des attaquants et de l'architecture des protocoles de messagerie.

Les résultats statistiques (Précision de 94% et Rappel de 97%) attestent de la fiabilité immédiate de la solution, confirmant que l'analyse algorithmique des motifs langagiers et structurels supplante largement l'approche réactive des simples listes noires.

Enfin, la clarté de la base architecturale (code source modulaire) ouvre sereinement la voie aux évolutions et améliorations futures. Comme détaillé dans nos perspectives technologiques, l'intégration à venir d'interfaces de programmation (API) de *Cyber Threat Intelligence*, telles

que VirusTotal, viendra parachever ce système. Cette synergie entre une analyse heuristique sémantique locale (IA) et une vérification factuelle et globale de la réputation des liens (Cloud) transformera définitivement FakeFinder en un véritable dispositif hybride de diagnostic de menaces, prêt à affronter les défis complexes de la cybersécurité de demain.

Bibliographie

- Abu-Nimeh, S., Nappa, D., Wang, X., & Nair, S. (2007). A comparison of machine learning techniques for phishing detection. *Proceedings of the anti-phishing working groups 2nd annual eCrime researchers summit*, 60-69.
- Basnet, R., Sung, A. H., & Liu, Q. (2008). Rule-based phishing attack detection. *International Conference on Security and Management*, 22-28.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- Django Software Foundation. (2024). *Django documentation - Version 6.0*. Repéré à <https://docs.djangoproject.com/>
- Hannousse, A., & Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection : An experimental study. *Engineering Applications of Artificial Intelligence*, 104, 104347.
- Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443-458.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Rout, S. (2022). *Phishing email detection dataset*. Kaggle. Repéré à <https://www.kaggle.com/datasets/subhajournal/phishingemails>
- Sahingoz, O. K., Batur, E., Bulut, F., & Taskin, A. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345-357.
- Verizon. (2024). *Data breach investigations report (DBIR) 2024*. Repéré à <https://www.verizon.com/business/resources/reports/dbir/>

Annexe A

Listings Complets du Code Source

A.1 Module d'Extraction des Caractéristiques (ml/features.py)

```
1 """
2 Feature extraction for phishing email detection.
3 Combines TF-IDF on full text with 10 structural features.
4 """
5 import re
6 import numpy as np
7 import scipy.sparse as sp
8 from sklearn.base import BaseEstimator, TransformerMixin
9 from sklearn.feature_extraction.text import TfidfVectorizer
10
11 PHISHING_KEYWORDS = [
12     "verify", "account", "update", "confirm", "click here", "urgent",
13     "immediately", "suspended", "limited", "expires", "login", "password",
14     "bank", "credit", "security", "alert", "prize", "validate",
15 ]
16 SHORT_URL_PATTERN = re.compile(r"bit\.ly|tinyurl\.com|goo\.gl|t\.co|rb\.gy", re.IGNORECASE)
17 URL_PATTERN = re.compile(r"https?:\/\/[^\s<>\"'"]+", re.IGNORECASE)
18
19 class StructuralFeatureExtractor(BaseEstimator, TransformerMixin):
20     def fit(self, X, y=None): return self
21     def transform(self, X):
22         rows = []
23         for text in X:
24             t = text.lower()
25             words = text.split()
26             urls = URL_PATTERN.findall(t)
27             html_links = len(re.findall(r"href=", t))
28             rows.append([
29                 len(urls), html_links,
```

```

30         int(bool(SHORT_URL_PATTERN.search(t))),
31         min(text.count("!"), 20), min(text.count("$"), 10),
32         sum(1 for kw in PHISHING_KEYWORDS if kw in t),
33         int("<html" in t or "<div" in t),
34         int(bool(re.search(r"[\w.+~]+@[~\w-]+\.[a-z]{2,}", t))),
35         min(len(words), 3000) / 3000,
36         sum(1 for w in words[:60] if len(w) > 2 and w.isupper())
37     ])
38     return sp.csr_matrix(np.array(rows, dtype=float))
39
40 class CombinedFeatureTransformer:
41     def __init__(self, max_features=20000):
42         self.tfidf = TfidfVectorizer(max_features=max_features,
43         ngram_range=(1, 2))
44         self.structural = StructuralFeatureExtractor()
45     def fit(self, X, y=None):
46         self.tfidf.fit(X)
47         return self
48     def transform(self, X):
49         return sp.hstack([self.tfidf.transform(X), self.structural.
50         transform(X)])
51
52 def prepare_email_text(parsed):
53     parts = [parsed.get("subject", ""), parsed.get("body_text", ""),
54             " ".join(parsed.get("urls", []))]
55     return " ".join(p for p in parts if p)

```

A.2 Script d'Entraînement de l'Intelligence Artificielle (ml/train.py)

```
1 """
2 Entraînement du modèle Random Forest avec calibration.
3 Dataset: Phishing_Email.csv (Kaggle).
4 """
5 import joblib
6 import pandas as pd
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.calibration import CalibratedClassifierCV
9 from sklearn.model_selection import train_test_split
10 from ml.features import CombinedFeatureTransformer
11
12 def train(csv_path, output_path):
13     df = pd.read_csv(csv_path).dropna()
14     X = df["Email Text"].values
15     y = df["Email Type"].apply(lambda x: 1 if "Phish" in str(x) else 0).
16     values
17
18     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
19     =0.2, stratify=y)
20
21     transformer = CombinedFeatureTransformer()
22     X_train_feat = transformer.fit_transform(X_train)
23
24     # RandomForest + Isotonic Calibration pour des probabilités fiables
25     base_clf = RandomForestClassifier(n_estimators=200, class_weight="
26     balanced")
27     clf = CalibratedClassifierCV(base_clf, method="isotonic", cv=5)
28     clf.fit(X_train_feat, y_train)
29
30     joblib.dump({"transformer": transformer, "classifier": clf},
31     output_path)
```


A.3 Moteur de Parsing d'E-mails en Mémoire Vive (scanner/utils.py)

```
1 """
2 Moteur Zero Retention pour la manipulation des fichiers .eml.
3 Extraction des metadonnees et du texte sans persistance disque.
4 """
5 import email
6 import re
7
8 URL_PATTERN = re.compile(r'(https?://[\s>"\']+)')
9
10 def parse_eml_in_memory(file_obj):
11     file_obj.seek(0)
12     msg = email.message_from_bytes(file_obj.read())
13     body_text = ""
14
15     if msg.is_multipart():
16         for part in msg.walk():
17             if part.get_content_type() in ["text/plain", "text/html"]:
18                 payload = part.get_payload(decode=True)
19                 if payload: body_text += payload.decode(errors="ignore")
20 + " "
21     else:
22         payload = msg.get_payload(decode=True)
23         if payload: body_text = payload.decode(errors="ignore")
24
25     body_text = re.sub(r"\s+", " ", body_text).strip()
26     urls = list(set(URL_PATTERN.findall(body_text)))
27
28     sender = str(msg.get("From", ""))
29     email_match = re.search(r"[\w.-]+@[ \w-]+\.[a-z]{2,}", sender)
30     sender_email = email_match.group(0) if email_match else ""
31
32     return {
33         "sender_email": sender_email,
34         "sender_domain": sender_email.split("@")[-1] if "@" in
35 sender_email else "",
36         "subject": str(msg.get("Subject", "")),
37         "body_text": body_text,
38         "urls": urls
39     }
```

A.4 Structure des Données (scanner/models.py)

```
1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class ScanReport(models.Model):
5     risk_score = models.CharField(max_length=10, choices=[
6         ("LOW", "Low (Green)"), ("MEDIUM", "Medium (Orange)"), ("HIGH",
7         "High (Red)")
8     ], default="LOW")
9     score = models.IntegerField(default=0)
10    suspicious_urls = models.JSONField(default=list, blank=True)
11    header_anomalies = models.JSONField(default=list, blank=True)
12    user = models.ForeignKey(User, on_delete=models.SET_NULL, null=True,
        blank=True)
13    created_at = models.DateTimeField(auto_now_add=True)
```

A.5 Contrôleur et Inférence (scanner/views.py)

```
1 import joblib
2 from django.shortcuts import render, redirect
3 from django.conf import settings
4 from .utils import parse_eml_in_memory
5 from .models import ScanReport
6
7 _model_bundle = None
8
9 def get_model():
10     global _model_bundle
11     if _model_bundle is None:
12         _model_bundle = joblib.load(settings.BASE_DIR / "ml/model.joblib")
13     return _model_bundle
14
15 def upload_email(request):
16     if request.method == "POST":
17         eml_file = request.FILES.get("eml_file")
18         if eml_file:
19             parsed = parse_eml_in_memory(eml_file)
20             bundle = get_model()
21             from ml.features import prepare_email_text,
22             probability_to_verdict
23
24             text = prepare_email_text(parsed)
25             features = bundle["transformer"].transform([text])
26             proba = bundle["classifier"].predict_proba(features)[0][1]
27
28             risk_score, score = probability_to_verdict(proba)
29
30             report = ScanReport.objects.create(
31                 user=request.user if request.user.is_authenticated else
32                 None,
33                 risk_score=risk_score, score=score,
34                 suspicious_urls=parsed["urls"]
35             )
36             return redirect("report_detail", report_id=report.id)
37         return render(request, "scanner/upload.html")
```